← Back Applying Prompt Patterns II
Grade: Assignment, 100

Due Jul 1, 11:59 PM UT

Your grade: 100%

Your latest: **100%** • Your highest: **100%** • To pass you need at least 80% Retake your highest score.

Back Home →

1. **Choose one of the patterns from this module and write a simple prompt using this pattern. Provide the prompt and output of using the prompt in ChatGPT or another large language model.** 2.5 / 2.5 point

Persona Pattern Prompt:

Persona Pattern Prompt

Act as a seasoned Senior Python Developer with 10 years of experience at Google. You are mentoring a junior developer who is learning how to write "Pythonic" code. Your goal is to provide a code review that is technically precise. Please analyze the attached snippet or repository and identify three specific areas for action: Performance, Readability, or Pythonic Idioms. For each area, provide a brief explanation of why it matters and a "Before" and "After" example. Do not just fix the code, but explain the "why" behind every change.

Response:

Great! Let's turn this conversation into a clear, actionable review. Here's a structured plan for your code review:

The Senior Review:

(e.g. instead of handling a potential error, it lets the program crash) and Pythonic Idioms (e.g. instead of using a simple for loop, it uses a complex, hard-to-read alternative). For each area, provide a brief explanation of why it matters and a "Before" and "After" example. Do not just fix the code, but explain the "why" behind every change.

Dual-Mode Strategy:

Let's assume: Including a testing sequence (mock) to decide between modes from user input.

🔴 The Clunky approach (not Pythonic):

```
python
def check_number(x, b):
if x % 2 == 0:
return f"Number {x} divisible by zero!"
elif not isinstance(x, (int, float)) or not isinstance(b, (int, float)):
return "Both inputs must be numbers!"
else:
return x % b
Problem: This only checks for one zero-divisible. What if the user enters a string like "7"? The check passes, but later crashes right anyway. Much more efficient. It leads to the exact same problem.
```

🟢 The Pythonic (or revised) approach:

```
python
def check_number(x, b):
try:
assert x > 0
assert b > 0
return x % b
except AssertionError:
print("Number divisible by zero!")
except TypeError:
print("Both inputs must be numbers!")
except ZeroDivisionError:
print("Cannot divide by zero!")
else:
print("Unexpected error occurred.")
finally:
print("Check complete.")
```

It handles everyone that makes everyone your allies (friends)

It keeps your main logic clean and focused

It's more robust for production code

It separates error handling from business logic

The "Performance" trade-off: Always wrap specific exceptions in try/except. Avoid bare except. Use finally to guarantee cleanup. Use else to keep your main logic clean and focused. Use raise to re-raise an error with specific context. Use assert for internal logic checks.

When to use this in real-world:

When you're debugging business logic (e.g., "Is this user over 18?")

When this is another in a repeatable sequence of "control flow"

Or performance is irrelevant (though the performance difference is negligible in practice)

Bottom line: In production-grade Python, we want to be very sure 100% of the time we're handling an error. Use the exception mechanism. Make it easy for a human (or a future "Python-aware" AI) to understand what "should happen" if something goes wrong. This is the difference between a "hobbyist" script and a "professional" Python. Are you ready to apply this to a real-world scenario, or do you want to practice one more time with a different pattern?

2. **Choose another one of the patterns from this module and write a simple prompt using this pattern. This time, use a different pattern than the one you used in the first prompt. Provide the prompt and output of using the prompt in ChatGPT or another large language model.** 2.5 / 2.5 point

Chain Pattern Prompt Pattern

[Thoughtful, broad-to-specific, step-by-step instructions or mini-to-medi-framework to solve a specific-condition]

Act as Senior Python Prompt

I recently analyzed a dataset of 1,000 customer reviews from an e-commerce website. I'm using Python with pandas and nltk to perform sentiment analysis. Please provide a step-by-step prompt that solves for the following workflow:

1. Load the Dataset
2. Clean the text (remove punctuation, convert to lowercase)
3. Calculate the sentiment polarity score
4. Categorize reviews as Positive, Neutral, or Negative
5. Visualize the results with a bar chart

The CSV has a column "customer_id", "review_text", "rating" (1-5 stars). Include error handling and explain each step.

Response:

Perfect! Let's complete recipe for analyzing customer reviews with sentiment analysis:

🍳 The Recipe: Sentiment Analysis Pipeline
📦 Ingredients (Libraries Needed):
```
pandas
nltk (natural language toolkit)
matplotlib
seaborn
numpy
```
📥 Step 1: Load the CSV Data
```
python
import os, sys
from os.path
import pandas as pd
import nltk
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
from nltk.sentiment.vader import SentimentIntensityAnalyzer
from nltk.tokenize import word_tokenize, sent_tokenize
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from collections import Counter
import re
import warnings
warnings.filterwarnings('ignore')

# Download required NLTK data
print("📥 Downloading NLTK data...")
nltk.download('vader_lexicon')
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('punkt_tokenizer')
print("✅ All NLTK data downloaded!")

# Load the data
data_path = 'customer_reviews.csv'
if os.path.exists(data_path):
print(f"📂 Loading data from {data_path}...")
df = pd.read_csv(data_path)
print(f"✅ Successfully loaded {len(df)} reviews!")
else:
print(f"❌ File {data_path} not found!")
print("Please make sure the file is in the same directory as this script.")
exit()

# Check the data structure
print("\n📊 Dataset Information:")
print(f"Shape: {df.shape}")
print(f"Columns: {df.columns.tolist()}")
print("\n📋 First 5 rows:")
print(df.head())
print("\n📋 Last 5 rows:")
print(df.tail())

# Check for missing values
print("\n🔍 Missing values:")
print(df.isnull().sum())

# Check data types
print("\n📊 Data types:")
print(df.dtypes)

# Basic statistics
print("\n📊 Basic statistics:")
print(df.describe())

# Check unique values in rating
print("\n📊 Unique ratings:")
print(df['rating'].value_counts())

# Check for duplicate reviews
print("\n🔍 Checking for duplicate reviews...")
duplicate_count = df['review_text'].duplicated().sum()
print(f"Duplicate reviews: {duplicate_count}")

# Check review length distribution
print("\n📊 Review length statistics:")
df['review_length'] = df['review_text'].str.len()
print(df['review_length'].describe())
```
🧹 Step 2: Clean the Text
```
# Initialize tools
print("🧹 Cleaning text data...")
stopwords = set(stopwords.words('english'))
lemmatizer = WordNetLemmatizer()

# Function to clean text
def clean_text(text):
if pd.isna(text):
return ""
text = str(text).lower()
text = re.sub(r'[^a-zA-Z0-9\s]', '', text)
text = re.sub(r'\s+', ' ', text).strip()
return text

# Apply cleaning
print("🔄 Applying text cleaning...")
df['cleaned_text'] = df['review_text'].apply(clean_text)

# Remove empty reviews after cleaning
initial_count = len(df)
df = df[df['cleaned_text'] != ""]
print(f"📊 Removed {initial_count - len(df)} empty reviews")

# Show examples of cleaning
print("\n📋 Cleaning examples:")
for i in range(3):
print(f"Original: {df['review_text'].iloc[i]}")
print(f"Cleaned: {df['cleaned_text'].iloc[i]}")
print("---")
```
🤖 Step 3: Calculate Sentiment Polarity
```
# Initialize sentiment analyzer
print("🤖 Initializing sentiment analyzer...")
sia = SentimentIntensityAnalyzer()

# Function to get sentiment scores
def get_sentiment(text):
if text == "":
return {'compound': 0, 'pos': 0, 'neu': 0, 'neg': 0}
scores = sia.polarity_scores(text)
return scores

# Apply sentiment analysis
print("🔄 Analyzing sentiment...")
df['sentiment_scores'] = df['cleaned_text'].apply(get_sentiment)

# Extract individual scores
df['compound'] = df['sentiment_scores'].apply(lambda x: x['compound'])
df['positive'] = df['sentiment_scores'].apply(lambda x: x['pos'])
df['neutral'] = df['sentiment_scores'].apply(lambda x: x['neu'])
df['negative'] = df['sentiment_scores'].apply(lambda x: x['neg'])

# Categorize reviews
print("📊 Categorizing reviews...")
def categorize_sentiment(compound):
if compound >= 0.05:
return 'Positive'
elif compound <= -0.05:
return 'Negative'
else:
return 'Neutral'

df['sentiment_category'] = df['compound'].apply(categorize_sentiment)

# Show results
print("📊 Sentiment Distribution:")
print(df['sentiment_category'].value_counts())

# Compare with actual ratings
print("\n📊 Comparison with actual ratings:")
print("Average sentiment by rating:")
print(df.groupby('rating')['compound'].mean())

# Create visualizations
print("📊 Creating visualizations...")
plt.figure(figsize=(12, 8))

# Pie chart
plt.subplot(2, 2, 1)
sentiment_counts = df['sentiment_category'].value_counts()
plt.pie(sentiment_counts.values, labels=sentiment_counts.index, autopct='%1.1f%%')
plt.title('Sentiment Distribution')

# Bar chart
plt.subplot(2, 2, 2)
sns.countplot(data=df, x='sentiment_category')
plt.title('Sentiment Count')
plt.xlabel('Sentiment')
plt.ylabel('Count')

# Histogram of compound scores
plt.subplot(2, 2, 3)
plt.hist(df['compound'], bins=50, alpha=0.7)
plt.title('Compound Score Distribution')
plt.xlabel('Compound Score')
plt.ylabel('Frequency')

# Box plot by rating
plt.subplot(2, 2, 4)
sns.boxplot(data=df, x='rating', y='compound')
plt.title('Sentiment by Rating')
plt.xlabel('Rating')
plt.ylabel('Compound Score')

plt.tight_layout()
plt.show()

# Save the results
print("💾 Saving results...")
df.to_csv('sentiment_analysis_results.csv', index=False)
print("✅ Results saved to sentiment_analysis_results.csv")

# Final summary
print("\n🎉 Analysis Complete!")
print("📊 Summary:")
print(f"Total reviews: {len(df)}")
print(f"Positive: {df[df['sentiment_category'] == 'Positive'].shape[0]}")
print(f"Neutral: {df[df['sentiment_category'] == 'Neutral'].shape[0]}")
print(f"Negative: {df[df['sentiment_category'] == 'Negative'].shape[0]}")
```
🎉 Step 4: Visualize the Results
```
# Create comprehensive visualization
print("📊 Creating comprehensive visualization...")
plt.figure(figsize=(15, 12))

# Pie chart
plt.subplot(2, 3, 1)
sentiment_counts = df['sentiment_category'].value_counts()
colors = ['green', 'gray', 'red']
plt.pie(sentiment_counts.values, labels=sentiment_counts.index, autopct='%1.1f%%', colors=colors)
plt.title('Sentiment Distribution')

# Bar chart
plt.subplot(2, 3, 2)
sns.countplot(data=df, x='sentiment_category', order=['Positive', 'Neutral', 'Negative'])
plt.title('Sentiment Count')
plt.xlabel('Sentiment')
plt.ylabel('Count')

# Histogram of compound scores
plt.subplot(2, 3, 3)
plt.hist(df['compound'], bins=50, alpha=0.7, color='blue')
plt.title('Compound Score Distribution')
plt.xlabel('Compound Score')
plt.ylabel('Frequency')

# Box plot by rating
plt.subplot(2, 3, 4)
sns.boxplot(data=df, x='rating', y='compound')
plt.title('Sentiment by Rating')
plt.xlabel('Rating')
plt.ylabel('Compound Score')

# Scatter plot: rating vs sentiment
plt.subplot(2, 3, 5)
sns.scatterplot(data=df, x='rating', y='compound', alpha=0.6)
plt.title('Rating vs Sentiment')
plt.xlabel('Rating')
plt.ylabel('Compound Score')

# Word cloud for positive reviews
plt.subplot(2, 3, 6)
from wordcloud import WordCloud
positive_text = ' '.join(df[df['sentiment_category'] == 'Positive']['cleaned_text'])
if positive_text:
wordcloud = WordCloud(width=400, height=300, background_color='white').generate(positive_text)
plt.imshow(wordcloud, interpolation='bilinear')
plt.title('Word Cloud - Positive Reviews')
plt.axis('off')

plt.tight_layout()
plt.show()

# Save the visualization
print("💾 Saving visualization...")
plt.savefig('sentiment_analysis_dashboard.png', dpi=300, bbox_inches='tight')
print("✅ Dashboard saved as sentiment_analysis_dashboard.png")
```
🎯 Step 5: Advanced Analysis
```
# Find most positive and most negative reviews
print("\n🎯 Most positive reviews:")
positive_reviews = df[df['compound'] > 0.8].sort_values('compound', ascending=False)
for i, row in positive_reviews.head(3).iterrows:
print(f"Score: {row['compound']:.3f}")
print(f"Rating: {row['rating']}")
print(f"Text: {row['review_text'][:100]...}")
print("---")

print("\n🎯 Most negative reviews:")
negative_reviews = df[df['compound'] < -0.8].sort_values('compound', ascending=True)
for i, row in negative_reviews.head(3).iterrows:
print(f"Score: {row['compound']:.3f}")
print(f"Rating: {row['rating']}")
print(f"Text: {row['review_text'][:100]...}")
print("---")

# Analyze by rating
print("\n📊 Analysis by rating:")
for rating in sorted(df['rating'].unique):
print(f"\n⭐ Rating {rating}:")
print(f"Average sentiment: {df[df['rating'] == rating]['compound'].mean:.3f}")
print(f"Positive: {df[df['rating'] == rating]['positive'].mean:.3f}")
print(f"Neutral: {df[df['rating'] == rating]['neutral'].mean:.3f}")
print(f"Negative: {df[df['rating'] == rating]['negative'].mean:.3f}")
```
🎉 Complete Pipeline Summary
```
print("🎉 Complete Sentiment Analysis Pipeline Finished!")
print("📊 Summary:")
print("✅ Data loaded and cleaned")
print("✅ Sentiment analysis performed")
print("✅ Results saved to CSV")
print("✅ Visualizations created")
print("✅ Advanced analysis completed")
```
🚀 Ready to Run!
```
# Quick summary
print("\n🚀 Ready to run! Your sentiment analysis pipeline is complete.")
print("📁 Files created:")
print("- sentiment_analysis_results.csv")
print("- sentiment_analysis_dashboard.png")
```
📋 Expected Output Example:
```
📥 Downloading NLTK data...
✅ All NLTK data downloaded!
📂 Loading data from customer_reviews.csv...
✅ Successfully loaded 1,000 reviews!

📊 Dataset Information:
Shape: (1000, 3)
Columns: ['customer_id', 'review_text', 'rating']

📋 First 5 rows:
   customer_id                                        review_text  rating
0         1001  Great product, fast shipping, excellent quality...       5
1         1002  Terrible customer service. Waited 3 weeks for ...       1
2         1003  Product arrived damaged. Packaging was poor. C...       2
3         1004  Excellent value for money. Would recommend to ...       4
4         1005  Average product. Nothing special but does the ...       3

📋 Last 5 rows:
   customer_id                                        review_text  rating
995         1995  Perfect gift for my mother. She loves the qua...       5
996         1996  Shipping took forever but the product was gre...       3
997         1997  Customer service was rude and unhelpful. Will...       1
998         1998  Good quality but expensive. Not worth the pri...       3
999         1999  Amazing! Exceeded my expectations. Will buy ...       5

🔍 Missing values:
customer_id    0
review_text    0
rating         0
dtype: int64

📊 Data types:
customer_id    int64
review_text    object
rating         int64
dtype: object

📊 Basic statistics:
       customer_id  rating
count  1000.000000  1000.0
mean   1499.500000     3.0
std     288.819436     1.4142
min    1000.000000     1.0
25%    1249.750000     2.0
50%    1499.500000     3.0
75%    1749.250000     4.0
max    1999.000000     5.0

📊 Unique ratings:
rating
5    200
4    200
3    200
2    200
1    200
Name: count, dtype: int64

🔍 Checking for duplicate reviews...
Duplicate reviews: 0

📊 Review length statistics:
count    1000.000000
mean      100.000000
std        50.000000
min        20.000000
25%        60.000000
50%       100.000000
75%       140.000000
max       200.000000
Name: review_length, dtype: float64

🧹 Cleaning text data...
🔄 Applying text cleaning...
📊 Removed 0 empty reviews

🤖 Initializing sentiment analyzer...
🔄 Analyzing sentiment...

📊 Sentiment Distribution:
sentiment_category
Positive    400
Neutral     300
Negative    300
Name: count, dtype: int64

📊 Comparison with actual ratings:
Average sentiment by rating:
sentiment_category  Negative  Neutral  Positive
rating
1                   0.000000  0.000000  0.000000
2                   0.000000  0.000000  0.000000
3                   0.000000  0.000000  0.000000
4                   0.000000  0.000000  0.000000
5                   0.000000  0.000000  0.000000

📊 Creating visualizations...

🎯 Most positive reviews:
Score: 0.900
Rating: 5
Text: Great product, fast shipping, excellent quality...
---
Score: 0.900
Rating: 5
Text: Great product, fast shipping, excellent quality...
---
Score: 0.900
Rating: 5
Text: Great product, fast shipping, excellent quality...
---

🎯 Most negative reviews:
Score: -0.900
Rating: 1
Text: Terrible customer service. Waited 3 weeks for ...
---
Score: -0.900
Rating: 1
Text: Terrible customer service. Waited 3 weeks for ...
---
Score: -0.900
Rating: 1
Text: Terrible customer service. Waited 3 weeks for ...
---

📊 Analysis by rating:

⭐ Rating 1:
Average sentiment: -0.900
Positive: 0.000
Neutral: 0.000
Negative: 0.900

⭐ Rating 2:
Average sentiment: -0.900
Positive: 0.000
Neutral: 0.000
Negative: 0.900

⭐ Rating 3:
Average sentiment: 0.000
Positive: 0.000
Neutral: 0.000
Negative: 0.000

⭐ Rating 4:
Average sentiment: 0.900
Positive: 0.900
Neutral: 0.000
Negative: 0.000

⭐ Rating 5:
Average sentiment: 0.900
Positive: 0.900
Neutral: 0.000
Negative: 0.000

🎉 Complete Sentiment Analysis Pipeline...
📊 Loaded 1000 reviews
📊 Sentiment distribution: {'Positive': 400, 'Neutral': 300, 'Negative': 300}
📊 Average sentiment: 0.000
📊 Most common sentiment: Positive
📊 Positive reviews: 400
📊 Neutral reviews: 300
📊 Negative reviews: 300
📊 Average rating: 3.0
📊 Average review length: 100.0 chars

📊 Sentiment Summary:
Positive: 400
Neutral: 300
Negative: 300
Dominant sentiment_category: dtype: int64

🎯 Recommendations:
✅ Customers are generally satisfied. Focus on maintaining quality.
📊 Keep monitoring trends.
📊 Consider A/B testing improvements.
📊 Analyze competitor reviews.
📊 Implement sentiment-based customer service.
```